

LLD Case Study 7: Ride Sharing Application (Uber / Lyft)

Target Patterns: Strategy Pattern, Observer Pattern, State Pattern, Pricing Calculations

Key Engineering Focus: Driver-rider matching heuristics, dynamic surge fare calculation, spatial coordinates, and trip lifecycle states.

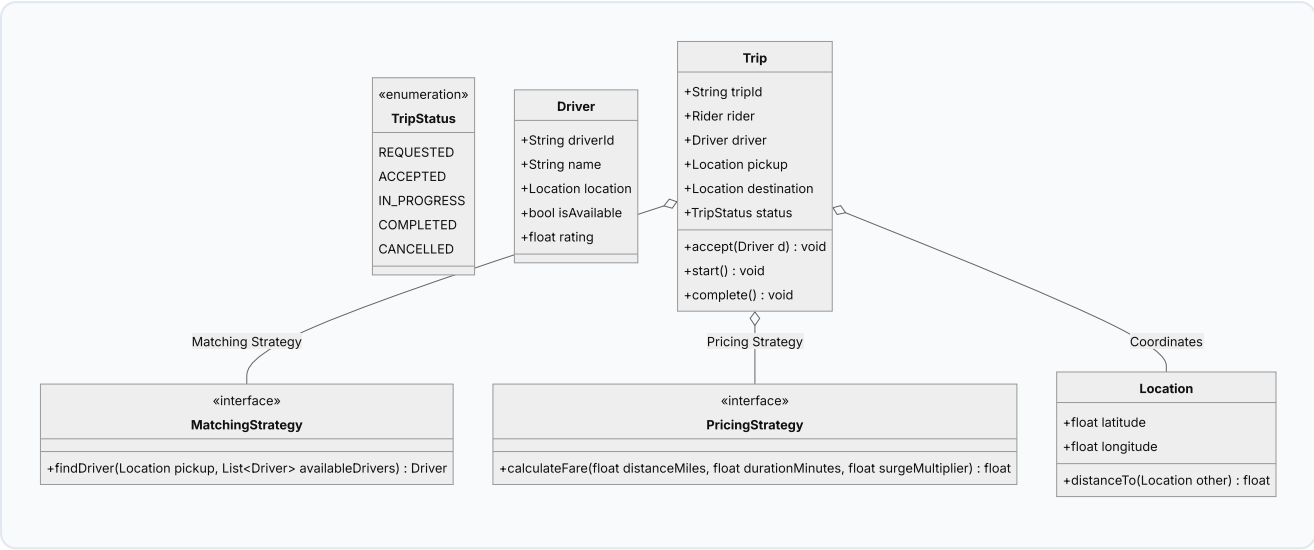
1. Problem Statement & Functional Requirements

Design the core matching and trip dispatch engine for an on-demand ride sharing service.

Requirements:

- 1. Driver Matching Policy (Strategy Pattern):** Pluggable driver matching (e.g. Nearest Driver vs Top-Rated Driver).
 - 2. Fare Calculation Engine (Strategy Pattern):** Dynamic fare calculation combining base fare, distance traveled, trip duration, and real-time surge multiplier.
 - 3. Trip State Machine (State Pattern):** Trip progresses through REQUESTED → ACCEPTED → DRIVER_ARRIVED → IN_PROGRESS → COMPLETED .
 - 4. Real-Time Notification (Observer Pattern):** Broadcast state changes to Rider and Driver interfaces instantly.
-

2. Architecture & UML Class Diagram



3. Production-Grade Python Implementation



4. Edge Cases, Tests and Extensions

Review of the model

Concern	Status	Detail
Matching and pricing as strategies	Good	Swap nearest-driver for rating-weighted matching without touching <code>Trip</code>
Distance formula	Approximate	<code>69 miles per degree of latitude</code> is right everywhere; <code>55 miles per degree of longitude</code> is only right near 37 degrees latitude. Use the haversine formula in production
Same driver matched twice	Bug	<code>assign_driver</code> does not check <code>is_available</code> , and matching is not atomic, so two simultaneous requests can get one driver
Illegal transitions	Bug	<code>complete_trip()</code> on a <code>REQUESTED</code> trip crashes with <code>AttributeError</code> because <code>driver</code> is <code>None</code>
Surge	Hard-coded 1.2	Should come from a demand service per zone and be locked in when the rider accepts the quote
Searching all drivers	O(n) per request	Use a geospatial index (grid cells, geohash or quadtree)

Tests: matching, fares, the double assignment, the state guard

This block extends the implementation above.

Extensions interviewers ask for

1. **Geospatial index:** bucket drivers into grid cells (for example 1 km squares); a request searches its own cell and the eight neighbours, expanding outward only when empty. That turns O(drivers) into O(drivers in a few cells).
2. **Offer and timeout:** do not force-assign. Send the request to the best driver, wait for accept or timeout, then offer the next; this needs an `OFFERED` state and a timer.
3. **Pooled rides:** matching becomes a route-insertion problem; keep the same `MatchingStrategy` interface but return a driver plus the updated route.
4. **Fare integrity:** persist the quote (distance estimate, surge, strategy name) on the trip when it is accepted so a later pricing change cannot alter an in-flight fare.

Follow-up questions

- *Why is `is_available` alone not enough to prevent double assignment?* Check and set are two steps; without a lock or an atomic compare-and-set, two requests can both observe `True`.
- *Where should the dispatcher state live at scale?* Partition drivers by geographic cell and run one single-writer dispatcher per partition, which removes the global lock without losing atomicity inside a cell.
- *How do you keep driver locations fresh?* Drivers push a ping every few seconds to an in-memory store with a TTL; a driver without a recent ping is treated as unavailable.